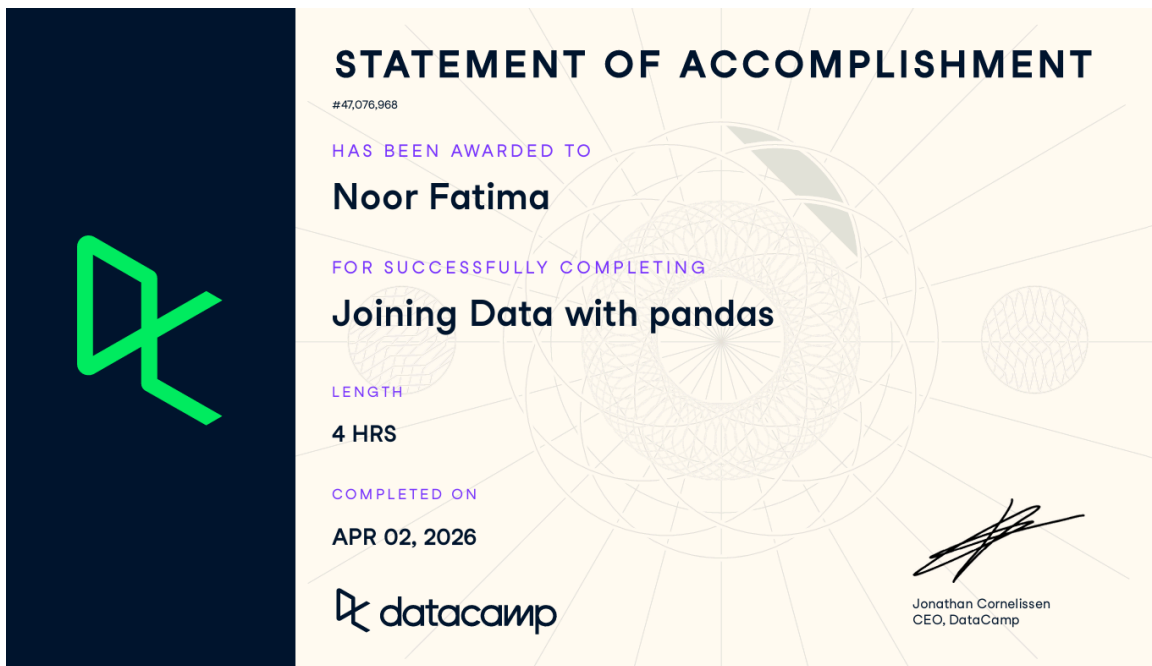




JOINING DATA WITH PANDAS:

INNER JOIN

In [1]: `import pandas as pd`

```
taxi_owners = pd.DataFrame({
    "vid": [1, 2, 3],
    "owner_name": ["Ali", "Sara", "Ahmed"]
})
taxi_veh = pd.DataFrame({
    "vid": [1, 2, 3],
    "fuel_type": ["Petrol", "Diesel", "Petrol"]
})
```

In [2]: `taxi_own_veh = taxi_owners.merge(taxi_veh, on="vid", suffixes=("_own", "_veh"))`
`print(taxi_own_veh.columns)`
`print(taxi_own_veh["fuel_type"].value_counts())`

```
Index(['vid', 'owner_name', 'fuel_type'], dtype='object')
fuel_type
Petrol    2
Diesel    1
Name: count, dtype: int64
```

In [3]: `taxi_own_veh = taxi_owners.merge(taxi_veh, on='vid', suffixes=('_own', '_veh'))`
`print(taxi_own_veh.columns)`

```
Index(['vid', 'owner_name', 'fuel_type'], dtype='object')
```

```
In [4]: taxi_own_veh = taxi_owners.merge(taxi_veh, on='vid', suffixes=('_own', '_veh'))
print(taxi_own_veh['fuel_type'].value_counts())
```

```
fuel_type
Petrol    2
Diesel    1
Name: count, dtype: int64
```

```
In [5]: import pandas as pd

wards = pd.DataFrame({
    "ward": [101, 102, 103],
    "ward_name": ["North", "South", "East"]
})
census = pd.DataFrame({
    "ward": [101, 102, 103],
    "population": [5000, 7000, 6000],
    "households": [1200, 1500, 1300]
})
```

```
In [6]: wards_census = wards.merge(census, on='ward')
print('wards_census table shape:', wards_census.shape)
```

```
wards_census table shape: (3, 4)
```

ONE TO MANY RELATIONSHIPS:

```
In [7]: import pandas as pd

licenses = pd.DataFrame({
    "account": [101, 102, 103, 104, 105],
    "title": ["Manager", "Clerk", "Manager", "Assistant", "Clerk"]
})
biz_owners = pd.DataFrame({
    "account": [101, 102, 103, 104, 105],
    "owner_name": ["Ali", "Sara", "Ahmed", "Zara", "Bilal"]
})
```

```
In [8]: licenses_owners = licenses.merge(biz_owners, on='account')

counted_df = licenses_owners.groupby('title').agg({'account': 'count'})

sorted_df = counted_df.sort_values(by='account', ascending=False)

print(sorted_df.head())
```

```
          account
title
Clerk           2
Manager         2
Assistant        1
```

MERGING MULTIPLE DATAFRAMES:

```
In [9]: import pandas as pd

ridership = pd.DataFrame({
    "year": [2023, 2023, 2023, 2023],
    "month": [1, 1, 1, 1],
    "day": [1, 2, 3, 4],
    "station_id": [101, 102, 103, 104],
    "rides": [250, 300, 200, 400]
})
cal = pd.DataFrame({
    "year": [2023, 2023, 2023, 2023],
    "month": [1, 1, 1, 1],
    "day": [1, 2, 3, 4],
    "weekday": ["Mon", "Tue", "Wed", "Thu"],
    "holiday": [False, False, False, False]
})
stations = pd.DataFrame({
    "station_id": [101, 102, 103, 104],
    "station_name": ["North Station", "South Station", "East Station", "West Station"],
    "capacity": [50, 40, 60, 30]
})
```

```
In [10]: ridership_cal_stations = ridership.merge(cal, on=['year','month','day']) \
        .merge(stations, on='station_id')

print(ridership_cal_stations)
```

	year	month	day	station_id	rides	weekday	holiday	station_name	\
0	2023	1	1	101	250	Mon	False	North Station	
1	2023	1	2	102	300	Tue	False	South Station	
2	2023	1	3	103	200	Wed	False	East Station	
3	2023	1	4	104	400	Thu	False	West Station	

	capacity
0	50
1	40
2	60
3	30

```
In [11]: import pandas as pd

licenses_owners = pd.DataFrame({
    "account": [101, 102, 103, 104, 105],
    "title": ["Manager", "Clerk", "Manager", "Assistant", "Clerk"]
})
counted_df = licenses_owners.groupby('title', as_index=False).agg(account_count=('a
sorted_df = counted_df.sort_values(by='account_count', ascending=False)

print(sorted_df.head())
```

	title	account_count
1	Clerk	2
2	Manager	2
0	Assistant	1

LEFT JOIN:

In [12]: `import pandas as pd`

```
movies = pd.DataFrame({
    "movie_id": [1, 2, 3, 4],
    "title": ["Movie A", "Movie B", "Movie C", "Movie D"],
    "genre": ["Action", "Comedy", "Action", "Drama"]
})
financials = pd.DataFrame({
    "movie_id": [1, 2, 3],
    "budget": [100000, 50000, 75000],
    "revenue": [200000, 80000, 120000]
})
```

In [13]: `movies_financials = movies.merge(financials, on="movie_id", how="left")`
`print(movies_financials)`

	movie_id	title	genre	budget	revenue
0	1	Movie A	Action	100000.0	200000.0
1	2	Movie B	Comedy	50000.0	80000.0
2	3	Movie C	Action	75000.0	120000.0
3	4	Movie D	Drama	NaN	NaN

In [14]: `import pandas as pd`

```
toy_story = pd.DataFrame({
    "movie_id": [1, 2, 3],
    "title": ["Toy Story 1", "Toy Story 2", "Toy Story 3"],
    "release_year": [1995, 1999, 2010]
})
taglines = pd.DataFrame({
    "movie_id": [1, 2],
    "tagline": ["The adventure begins.", "The toys are back!"]
})
```

In [15]: `toystory_tag = toy_story.merge(taglines, on="movie_id", how="left")`
`print(toystory_tag)`
`print(toystory_tag.shape)`

	movie_id	title	release_year	tagline
0	1	Toy Story 1	1995	The adventure begins.
1	2	Toy Story 2	1999	The toys are back!
2	3	Toy Story 3	2010	NaN

(3, 4)

OTHERS JOIN:

In [16]: `import pandas as pd`

```
action_movies = pd.DataFrame({
```

```

    "movie_id": [1, 2, 3],
    "title": ["Action A", "Action B", "Action C"],
    "budget": [100000, 200000, 150000]
})
scifi_movies = pd.DataFrame({
    "movie_id": [2, 3, 4],
    "title": ["Action B", "Action C", "SciFi D"],
    "revenue": [300000, 250000, 400000]
})

```

```

In [17]: action_scifi = action_movies.merge(scifi_movies, on="movie_id", how="right", suffixes=('_act', '_sci'))
print(action_scifi)

```

	movie_id	title_act	budget	title_sci	revenue
0	2	Action B	200000.0	Action B	300000
1	3	Action C	150000.0	Action C	250000
2	4	NaN	NaN	SciFi D	400000

```

In [18]: import pandas as pd
import matplotlib.pyplot as plt

movie_to_genres = pd.DataFrame({
    "id": [1, 2, 3, 4, 5, 6],
    "genre": ["Action", "Comedy", "Action", "Drama", "Comedy", "Sci-Fi"]
})
pop_movies = pd.DataFrame({
    "id": [2, 3, 4, 5, 6, 7],
    "title": ["Movie B", "Movie C", "Movie D", "Movie E", "Movie F", "Movie G"]
})

```

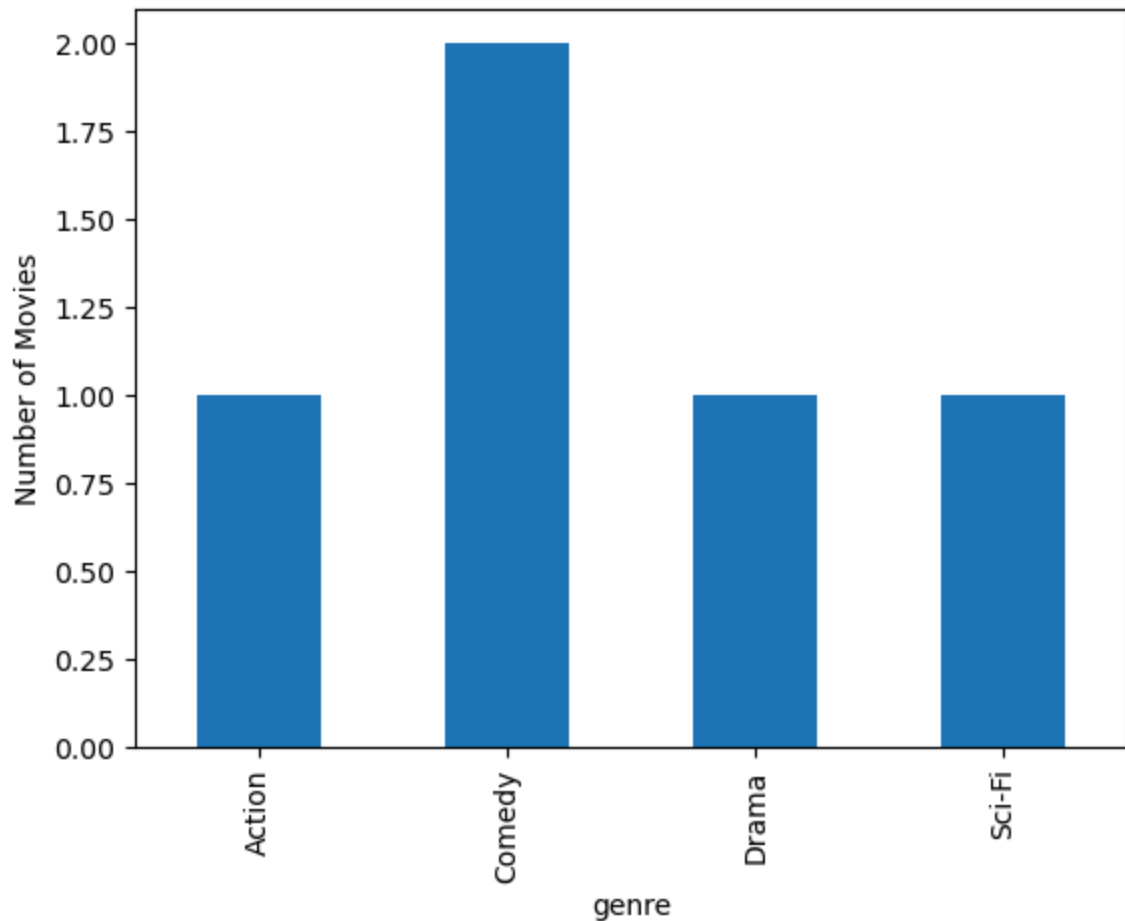
```

In [19]: genres_movies = movie_to_genres.merge(pop_movies, on='id', how='right')

genre_count = genres_movies.groupby('genre').agg({'id': 'count'})

genre_count.plot(kind='bar', legend=False)
plt.ylabel("Number of Movies")
plt.show()

```



MERGING A TABLE TO ITSELF:

In [20]: `import pandas as pd`

```
crews = pd.DataFrame({
    "crew_id": [1, 2, 3, 4],
    "name": ["Alice", "Bob", "Charlie", "David"],
    "role": ["Director", "Producer", "Director", "Producer"],
    "movie_id": [101, 101, 102, 102]
})
```

In [21]: `crews_self_merged = crews.merge(crews, on="movie_id", suffixes=("_left", "_right"))`
`print(crews_self_merged)`

	crew_id_left	name_left	role_left	movie_id	crew_id_right	name_right	\
0	1	Alice	Director	101	1	Alice	
1	1	Alice	Director	101	2	Bob	
2	2	Bob	Producer	101	1	Alice	
3	2	Bob	Producer	101	2	Bob	
4	3	Charlie	Director	102	3	Charlie	
5	3	Charlie	Director	102	4	David	
6	4	David	Producer	102	3	Charlie	
7	4	David	Producer	102	4	David	

	role_right
0	Director
1	Producer
2	Director
3	Producer
4	Director
5	Producer
6	Director
7	Producer

MERGING ON INDEXES:

In [22]: `import pandas as pd`

```
movies = pd.DataFrame({
    "title": ["Movie A", "Movie B", "Movie C", "Movie D"],
    "genre": ["Action", "Comedy", "Drama", "Sci-Fi"]
})
ratings = pd.DataFrame({
    "rating": [4.5, 3.8, 4.2, 4.0],
    "votes": [1200, 800, 950, 1100]
})
```

In [23]: `movies_ratings = movies.merge(ratings, left_index=True, right_index=True)`

`print(movies_ratings.head())`

	title	genre	rating	votes
0	Movie A	Action	4.5	1200
1	Movie B	Comedy	3.8	800
2	Movie C	Drama	4.2	950
3	Movie D	Sci-Fi	4.0	1100

In [24]: `import pandas as pd`

```
sequels = pd.DataFrame({
    "title": ["Movie A2", "Movie B2", "Movie C2", "Movie D2"],
    "year": [2001, 2003, 2005, 2007]
})
financials = pd.DataFrame({
    "budget": [50000, 60000, 70000, 80000],
    "revenue": [150000, 180000, 200000, 220000]
})
```

```
In [25]: sequels_fin = sequels.merge(financials, left_index=True, right_index=True)

print(sequels_fin)
```

	title	year	budget	revenue
0	Movie A2	2001	50000	150000
1	Movie B2	2003	60000	180000
2	Movie C2	2005	70000	200000
3	Movie D2	2007	80000	220000

CONCATENATE DATAFRAMES TOGETHER VERTICALLY:

```
In [26]: import pandas as pd

album1_tracks = pd.DataFrame({
    "tid": [101, 102],
    "track_name": ["Track A1", "Track A2"],
    "duration": [210, 195]
})
album2_tracks = pd.DataFrame({
    "tid": [201, 202],
    "track_name": ["Track B1", "Track B2"],
    "duration": [230, 220]
})
album3_tracks = pd.DataFrame({
    "tid": [301],
    "track_name": ["Track C1"],
    "duration": [205]
})
```

```
In [27]: tracks_from_albums = pd.concat([album1_tracks, album2_tracks, album3_tracks], sort=

print(tracks_from_albums)
```

	duration	tid	track_name
0	210	101	Track A1
1	195	102	Track A2
0	230	201	Track B1
1	220	202	Track B2
0	205	301	Track C1

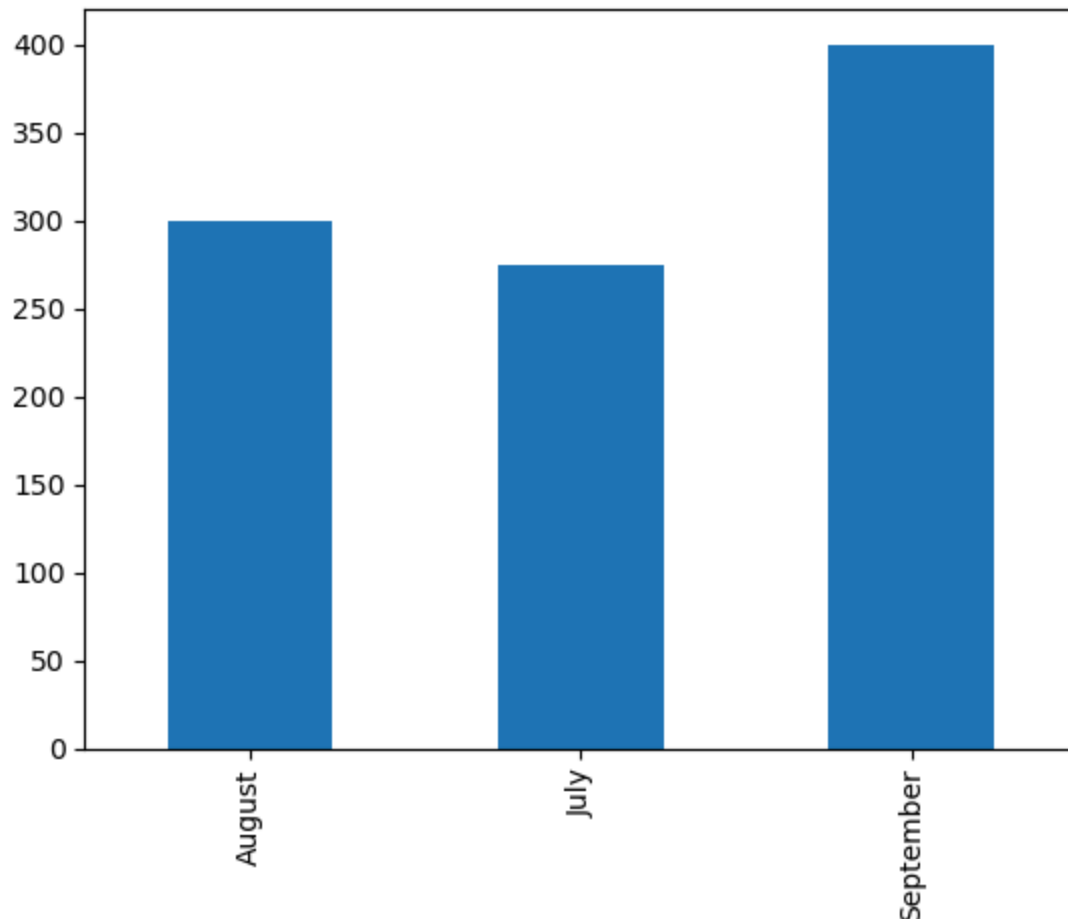
```
In [28]: import pandas as pd
import matplotlib.pyplot as plt

jul_invoices = pd.DataFrame({
    "invoice_id": [101, 102],
    "total": [250, 300]
})
aug_invoices = pd.DataFrame({
    "invoice_id": [103, 104],
    "total": [200, 400]
})
sep_invoices = pd.DataFrame({
```



```
"invoice_id": [105, 106],  
"total": [350, 450]  
})
```

```
In [29]: inv_jul_thr_sep = pd.concat([jul_invoices, aug_invoices, sep_invoices],  
                                     keys=['July', 'August', 'September'])  
avg_inv_by_month = inv_jul_thr_sep.groupby(level=0).agg({'total': 'mean'})  
avg_inv_by_month.plot(kind='bar', legend=False)  
plt.show()
```



VERIFYING INTEGRITY:

```
In [30]: import pandas as pd  
  
classic_2018 = pd.DataFrame({  
    "track_id": [1, 2],  
    "track_name": ["Classic A", "Classic B"],  
    "duration": [210, 195]  
})  
classic_2019 = pd.DataFrame({  
    "track_id": [3, 4],  
    "track_name": ["Classic C", "Classic D"],  
    "duration": [200, 220]  
})  
pop_2018 = pd.DataFrame({
```

```

        "track_id": [5, 6],
        "track_name": ["Pop A", "Pop B"],
        "duration": [180, 190]
    })
    pop_2019 = pd.DataFrame({
        "track_id": [7, 8],
        "track_name": ["Pop C", "Pop D"],
        "duration": [175, 185]
    })

```

```

In [31]: classic_18_19 = pd.concat([classic_2018, classic_2019], ignore_index=True)
        pop_18_19 = pd.concat([pop_2018, pop_2019], ignore_index=True)

```

USING MERGE_ORDERED():

```

In [32]: import pandas as pd

        gdp = pd.DataFrame({
            "year": [2018, 2019, 2020],
            "gdp_value": [20000, 21000, 19000]
        })
        sp500 = pd.DataFrame({
            "date": [2018, 2019, 2020],
            "sp500_value": [2700, 2900, 3200]
        })

```

```

In [33]: gdp_sp500 = pd.merge_ordered(gdp, sp500, left_on='year', right_on='date', how='outer')
        print(gdp_sp500)

```

	year	gdp_value	date	sp500_value
0	2018	20000	2018	2700
1	2019	21000	2019	2900
2	2020	19000	2020	3200

```

In [34]: gdp_sp500 = pd.merge_ordered(gdp, sp500,
                                     left_on='year', right_on='date',
                                     how='left', fill_method='ffill')

```

```

In [35]: import pandas as pd
        import matplotlib.pyplot as plt

        inflation = pd.DataFrame({
            "year": [2018, 2019, 2020],
            "cpi": [2.1, 1.8, 1.2]
        })
        unemployment = pd.DataFrame({
            "year": [2018, 2019, 2020],
            "unemployment_rate": [4.0, 3.8, 8.1]
        })

```

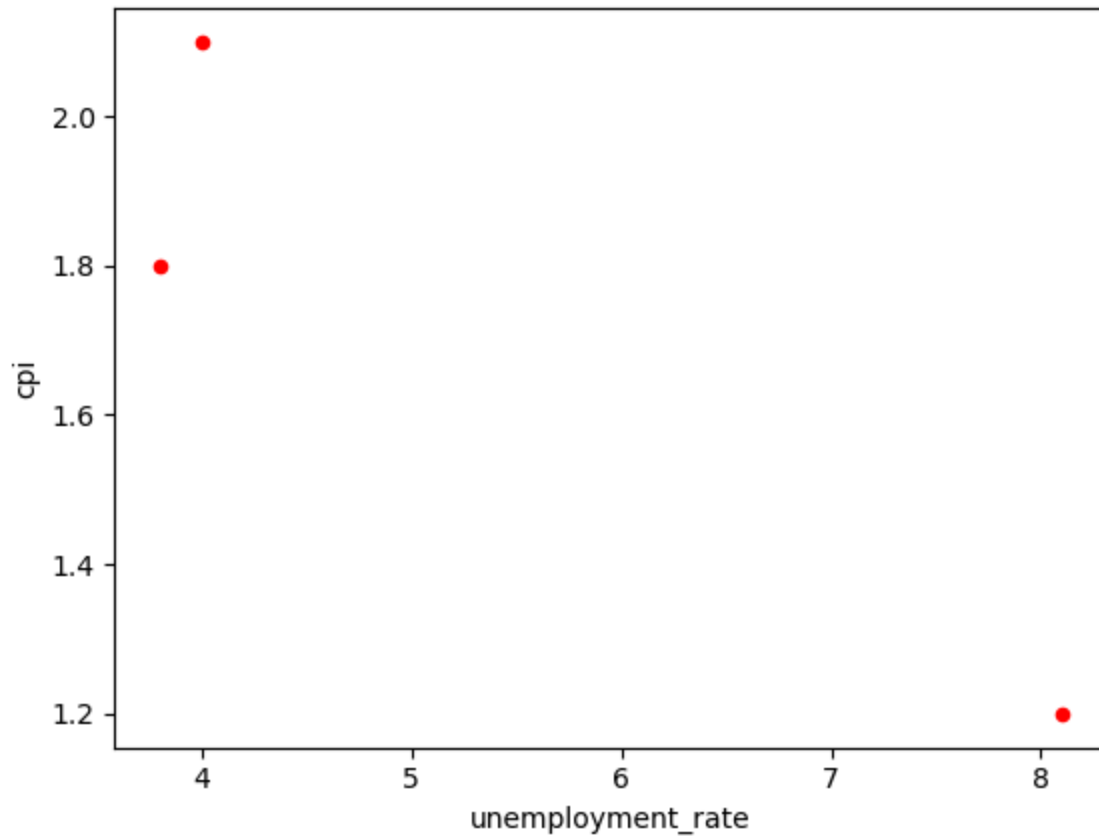
```

In [36]: inflation_unemploy = pd.merge_ordered(inflation, unemployment, on='year', how='inner')
        print(inflation_unemploy)

```

```
inflation_unemploy.plot(kind='scatter', x='unemployment_rate', y='cpi', color='red')
plt.show()
```

	year	cpi	unemployment_rate
0	2018	2.1	4.0
1	2019	1.8	3.8
2	2020	1.2	8.1



In [37]: `import pandas as pd`

```
gdp = pd.DataFrame({
    "country": ["USA", "USA", "Canada", "Canada"],
    "date": [2018, 2019, 2018, 2019],
    "gdp_value": [20000, 21000, 1700, 1800]
})
pop = pd.DataFrame({
    "country": ["USA", "USA", "Canada", "Canada"],
    "date": [2018, 2020, 2018, 2019], # Notice 2020 missing in GDP for USA
    "pop_value": [327, 331, 37, 38]
})
```

In [38]: `ctry_date = pd.merge_ordered(gdp, pop, on=['country', 'date'], how='outer', fill_met`
`print(ctry_date)`

	country	date	gdp_value	pop_value
0	Canada	2018	1700	37
1	Canada	2019	1800	38
2	USA	2018	20000	327
3	USA	2019	21000	327
4	USA	2020	21000	331

USING MERGE_asof():

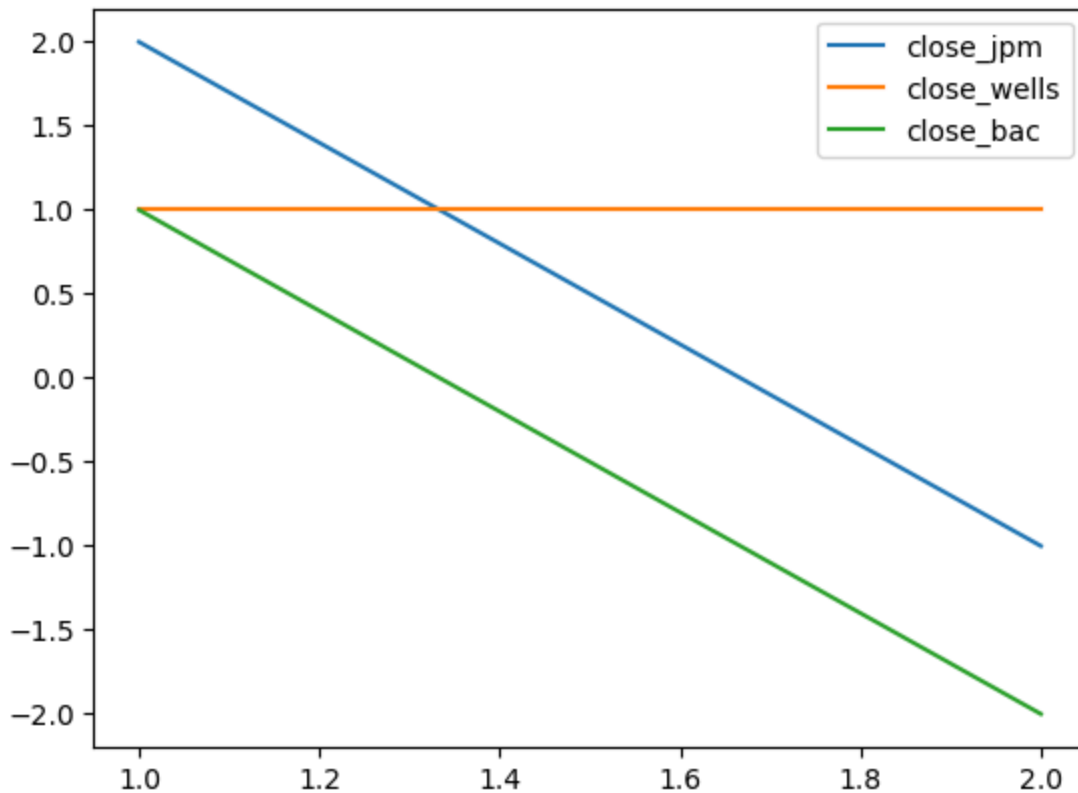
```
In [39]: import pandas as pd
import matplotlib.pyplot as plt

jpm = pd.DataFrame({
    "date": pd.to_datetime(["2021-01-01", "2021-01-02", "2021-01-03"]),
    "close_jpm": [120, 122, 121]
})
wells = pd.DataFrame({
    "date": pd.to_datetime(["2021-01-01", "2021-01-02", "2021-01-03"]),
    "close_wells": [30, 31, 32]
})
bac = pd.DataFrame({
    "date": pd.to_datetime(["2021-01-01", "2021-01-02", "2021-01-03"]),
    "close_bac": [28, 29, 27]
})
```

```
In [40]: jpm_wells = pd.merge_asof(jpm.sort_values('date'),
                                   wells.sort_values('date'),
                                   on='date')

jpm_wells_bac = pd.merge_asof(jpm_wells.sort_values('date'),
                              bac.sort_values('date'),
                              on='date')

price_diffs = jpm_wells_bac.diff()
price_diffs.plot(y=['close_jpm', 'close_wells', 'close_bac'])
plt.show()
```



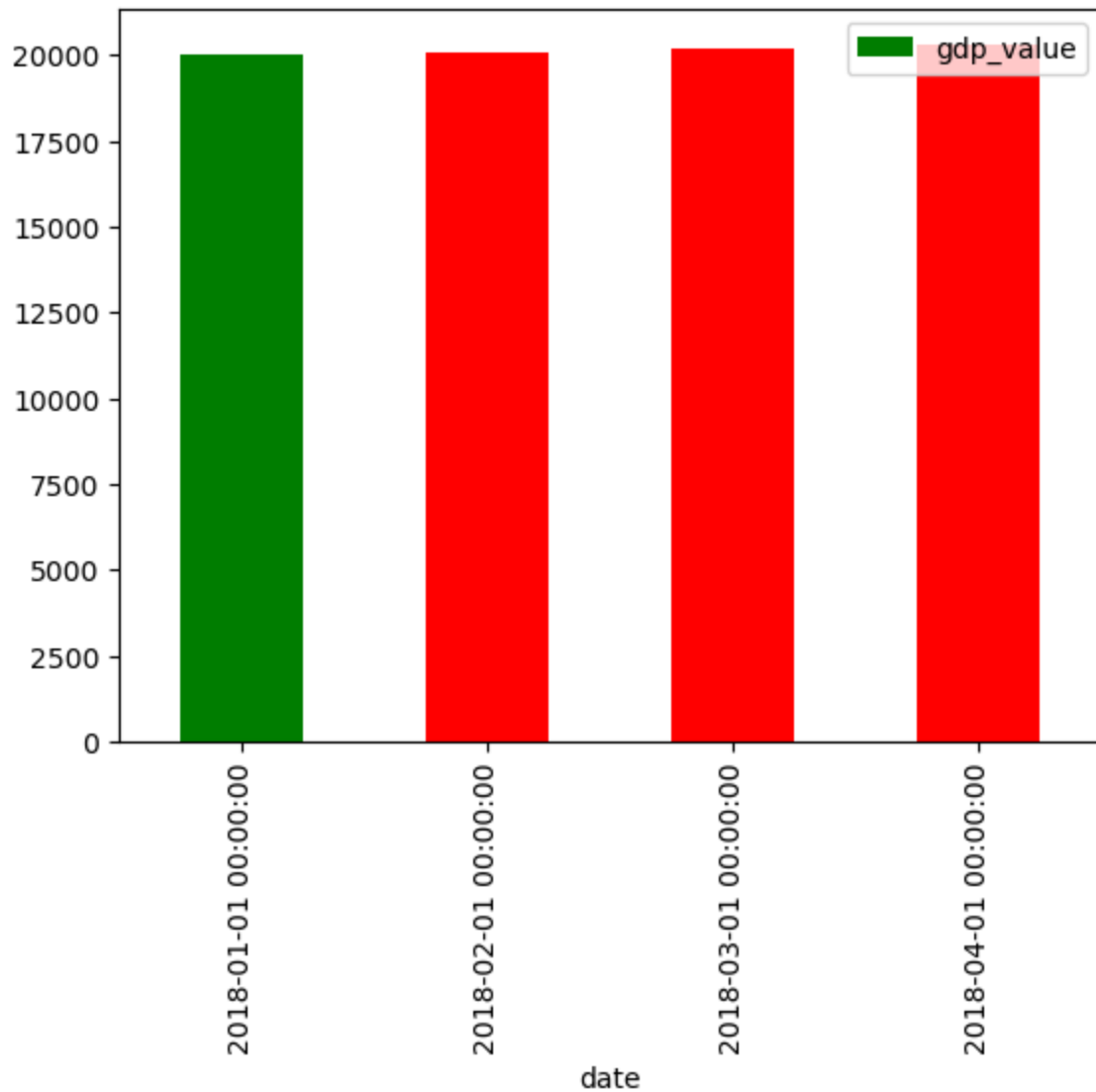
```
In [41]: import pandas as pd
import matplotlib.pyplot as plt

gdp = pd.DataFrame({
    "date": pd.to_datetime(["2018-01-01", "2018-02-01", "2018-03-01", "2018-04-01"]),
    "gdp_value": [20000, 20100, 20200, 20300]
})
recession = pd.DataFrame({
    "date": pd.to_datetime(["2018-02-01", "2018-04-01"]),
    "econ_status": ["recession", "recession"]
})

In [42]: gdp_recession = pd.merge_asof(gdp.sort_values('date'),
                                         recession.sort_values('date'),
                                         on='date',
                                         direction='backward')

is_recession = ['red' if s=='recession' else 'green' for s in gdp_recession['econ_s

gdp_recession.plot(kind='bar', y='gdp_value', x='date', color=is_recession, rot=90)
plt.show()
```



SELECTING DATA WITH .QUERY():

In [43]: `import pandas as pd`

```
gdp = pd.DataFrame({
    "country": ["USA", "USA", "Canada", "Canada"],
    "date": [2018, 2019, 2018, 2019],
    "gdp_value": [20000, 21000, 1700, 1800]
})
pop = pd.DataFrame({
    "country": ["USA", "USA", "Canada", "Canada"],
    "date": [2018, 2020, 2018, 2019],
    "pop_value": [327, 331, 37, 38]
})
```

In [44]: `gdp_pop = pd.merge_ordered(gdp, pop, on=['country','date'], how='outer', fill_metho
print(gdp_pop)`

	country	date	gdp_value	pop_value
0	Canada	2018	1700	37
1	Canada	2019	1800	38
2	USA	2018	20000	327
3	USA	2019	21000	327
4	USA	2020	21000	331

RESHAPING DATA WITH .MELT():

```
In [45]: import pandas as pd
import matplotlib.pyplot as plt
```

```
unempl = pd.DataFrame({
    "year": [2018, 2019],
    "Jan": [4.0, 3.8],
    "Feb": [3.9, 3.7],
    "Mar": [3.8, 3.6]
})
```

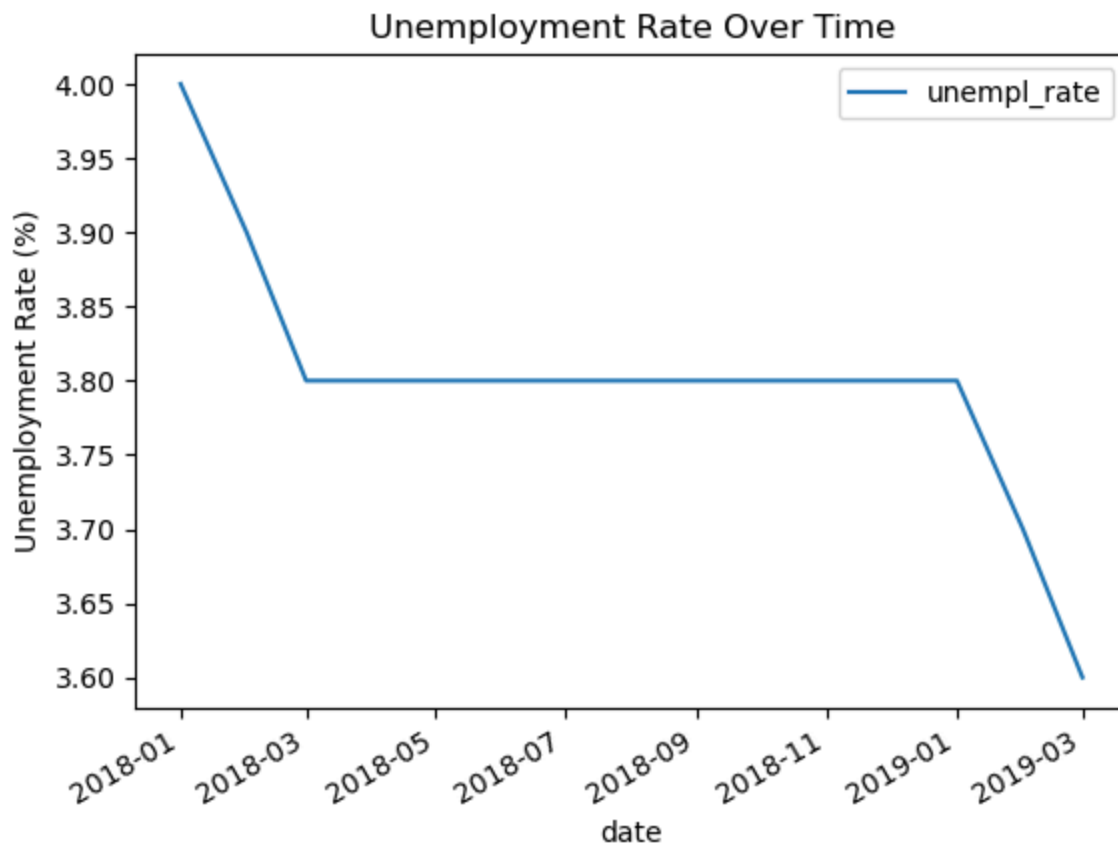
```
In [46]: ur_tall = unempl.melt(id_vars='year', var_name='month', value_name='unempl_rate')

month_map = {'Jan':1, 'Feb':2, 'Mar':3}
ur_tall['month_num'] = ur_tall['month'].map(month_map)

ur_tall['date'] = pd.to_datetime(dict(year=ur_tall['year'], month=ur_tall['month_num']))

ur_sorted = ur_tall.sort_values('date')
```

```
In [47]: ur_sorted.plot(kind='line', x='date', y='unempl_rate')
plt.title("Unemployment Rate Over Time")
plt.ylabel("Unemployment Rate (%)")
plt.show()
```



In [48]: `import pandas as pd`
`import matplotlib.pyplot as plt`

```
ten_yr = pd.DataFrame({
    "date": pd.to_datetime(["2020-01-01", "2020-01-02", "2020-01-03"]),
    "open": [1.5, 1.6, 1.55],
    "close": [1.55, 1.65, 1.60]
})
dji = pd.DataFrame({
    "date": pd.to_datetime(["2020-01-01", "2020-01-02", "2020-01-03"]),
    "close_dow": [28000, 28200, 28100]
})
```

In [49]: `bond_perc = ten_yr.melt(id_vars='date', var_name='metric', value_name='close_bond')`
`bond_perc_close = bond_perc.query("metric == 'close'")`
`dow_bond = pd.merge_ordered(dji, bond_perc_close, on='date', how='inner')`
`dow_bond.plot(y=['close_dow', 'close_bond'], x='date', rot=90)`
`plt.show()`

